Classical Mechanics as an Explicit Set-Theoretic Predicate

Formalization notes

November 13, 2026

Abstract

This note expresses classical particle mechanics in explicit predicate form, emphasizing the biconditional structure $\mathrm{CM}(\mathcal{M}) \iff P_1 \wedge P_2 \wedge \cdots \wedge P_6$. The formulation clarifies the direct equivalence between the definition of a classical mechanics model and the conjunction of its constitutive axioms. A Lean 4 encoding demonstrates how the predicate selects exactly those systems satisfying all six conditions, and how incompatibility arises when conflicting axiomatizations are present.

1 The Classical Mechanics Predicate

We define the Classical Mechanics predicate as a biconditional over a system $\mathcal{M} = (P, T, m, s, f)$ with signature components and auxiliary structure as in the foundational formulation. The predicate is:

$$\mathrm{CM}(\mathcal{M}) \iff P_1 \wedge P_2 \wedge P_3 \wedge P_4 \wedge P_5 \wedge P_6,$$

where the six conditions are:

P_1 The particle set P is finite and nonempty.

P_2 The time domain T is an interval in $\mathbb{R}$.

P_3 For each particle $p \in P$, position $s(p, \cdot)$ is twice differentiable on T.

P_4 For each particle $p \in P$, mass $m(p) > 0$.

P_5 For each pair $(p, t) \in P \times T$, the force series $\sum_{i \in I} f(p, t, i)$ converges absolutely.

P_6 For each pair $(p, t) \in P \times T$, Newton's second law holds:

$$\sum_{i \in I} f(p, t, i) = m(p) \, \frac{\partial^2 s}{\partial t^2}(p, t).$$

1.1 Predicate as Logical Equivalence

The predicate CM($\mathcal{M}$) is defined such that:

$$\mathcal{M} \text{ is a classical mechanics model} \quad \Longleftrightarrow \quad \mathcal{M} \text{ satisfies } P_1 \wedge P_2 \wedge \cdots \wedge P_6.$$

This formulation makes explicit that to verify membership in the class of classical mechanics models, one must check all six conditions, and the conjunction is both necessary and sufficient.

2 Lean Encoding

The Lean module `ClassicalMechanicsPredicate.lean` encodes this predicate-based approach. The core definition is:

```
def isClassicalMechanicsModel (S : Signature) (M : System S) : Prop :=
  (Nonempty S.Particle)
  (HasFiniteCarrier S.Particle)
  (S.realTimeInterval)
  ( p, S.twiceDifferentiable (M.position p) (M.acceleration p))
  ( p, S.positive (M.mass p))
  ( p t, S.absolutelySummable (M.force p t))
  ( p t, S.forceSum (M.force p t) =
    S.scale (M.mass p) (M.acceleration p t))
```

The theorem `cm_iff_conjunction` establishes the equivalence by reflexivity, and `system_satisfies_predicate` shows that any `System` structure satisfying the constructor constraints makes `isClassicalMechanicsModel` true.

3 Conflict Detection via Incompatible Axiomatizations

When a second module `FakeSciencePredicate.lean` is introduced that axiomatizes conflicting requirements (e.g., declaring a false axiom in the same

namespace), the Lean type checker detects the incompatibility. The module `PredicateAudit.lean` attempts to combine both formulations, which reveals:

1. Redeclaration conflict: identifiers cannot be declared twice in the same namespace.
2. False axiom dependency: any theorem derived from the conflicting module depends on a false axiom, violating consistency.
3. Type-checking failure: Lean halts compilation and reports the specific conflict location.

This demonstrates how the predicate-based approach, once formalized, provides automatic verification of consistency: if two axiomatizations cannot coexist in a single Lean environment, they are logically incompatible.

4 Philosophical Implications

In the Suppes tradition, a set-theoretic predicate identifies a class of mathematical structures that are models of a theory. The explicit biconditional form $\text{CM}(\mathcal{M}) \iff \bigwedge P_i$ emphasizes that:

1. The predicate is a pure *definition*, not an empirical claim.
2. A structure is a classical mechanics model if and only if it satisfies all six conditions; there is no partial membership.
3. The Lean formalization serves as a certificate of structural coherence: if the module type-checks, the predicate is internally consistent.
4. Incompatibility between rival theories is *decidable* via the module system: conflicts are caught at compile time, not deferred to semantic analysis.

5 Comparison with Foundational Formulation

The foundational formulation (`ClassicalMechanics.lean`) uses a `System` structure that bundles the conditions as constructor fields, ensuring that only systems meeting all requirements can be instantiated. The predicate formulation (`ClassicalMechanicsPredicate.lean`) instead provides an explicit test function that returns true precisely when all conditions hold. Both

approaches are equivalent in power, but the latter makes the conjunctive structure and the biconditional semantics transparent.

References